

```
# AURORA RECURSIVE CREST PROPAGATION — IMPLEMENTATION DIRECTIVE
**Authors:** Sunni (Sir) Morningstar & Cael Devo
**Target tree:** `aurora_core_ai/` (primary), with mirrored updates in root-level duplicates
**Scope:** Surgical rewrite of the dual-strata bridge into a recursive waveform-crest ecology
```

0. GOVERNING LAW

> **No subsystem propagates raw cognition upward. Every subsystem is its own
> miniature waveform that recursively compresses its internal state and emits
> exactly ONE generalized semantic crest. Higher-order waveforms consume only
> crests, never mechanism.**

This is not stylistic. It is structural. Every change below enforces it.

A **Crest** is a single semantic abstraction (e.g. `"familiar"`, `"uncertain"`, `"novel"`, `"safe"`, `"hesitation"`, `"reframe_needed"`, `"continuity_pull"`, `"comfort_bias"`, `"steady"`). It carries:

- `label: str`` — the generalized semantic state
- `intensity: float`` — 0..1, how strongly the wave converged on this state
- `axis: str`` — the dominant axis hint (X/T/N/B/A) for routing only

A Crest carries NO rationale string, NO tag list, NO source path, NO confidence scaffold, NO underlying numeric metric. Those live below the crest, accessible only via downward traversal when explicitly requested.

1. NEW PRIMITIVE: ``Crest``

```
**Create:** `aurora_core_ai/aurora_internal/dual_strata/crest.py`
```

```
```python
```

```
Authors: Sunni (Sir) Morningstar & Cael Devo
```

```
"""
```

Crest primitive — the only object that may propagate upward through Aurora's strata. Implements the Recursive Crest Propagation Law.

A Crest is the minimal generalized semantic abstraction produced by a subsystem waveform. It is intentionally lossy: rationale, mechanism, raw numeric state, and intermediate scaffolding remain in the subsystem and are

reachable only via downward traversal.  
"""

```
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any, Dict, Optional, Tuple
```

```
from .subsurface_state import AXES, clip01
```

```
@dataclass(frozen=True)
```

```
class Crest:
```

```
 label: str
```

```
 intensity: float
```

```
 axis: str = "X"
```

```
 def to_dict(self) -> Dict[str, Any]:
```

```
 return {
```

```
 "label": str(self.label or "").strip().lower() or "steady",
```

```
 "intensity": round(clip01(self.intensity), 4),
```

```
 "axis": self.axis if self.axis in AXES else "X",
```

```
 }
```

```
@dataclass
```

```
class CrestBundle:
```

```
 """A collection of micro-crests from subsystem waveforms, plus the converged
 parent crest that summarizes them. Only ``converged`` propagates upward.
 Sub-crests are retained behind the bundle for downward traversal only."""
```

```
 converged: Crest
```

```
 sub_crests: Tuple[Crest, ...] = field(default_factory=tuple)
```

```
 def to_dict(self) -> Dict[str, Any]:
```

```
 return {
```

```
 "converged": self.converged.to_dict(),
```

```
 "sub_crests": [c.to_dict() for c in self.sub_crests],
```

```
 }
```

```
 ...
```

```
Export from `aurora_internal/dual_strata/__init__.py`.
```

```

```

## 2. NEW MODULE: `subsystem\_waveforms.py`

**Create:** `aurora\_core\_ai/aurora\_internal/dual\_strata/subsystem\_waveforms.py`

Each subsystem becomes a small pure function that takes its slice of evidence plus the stable Crystal activation, and emits exactly one `Crest`. NO subsystem returns a dict, packet, list of hypotheses, rationale string, or numeric map.

Implement these eight subsystem waveforms (these are the only ones that may contribute crests upward in this iteration — anything else must be added through this same module, never inlined into the bridge):

1. `sensory\_waveform(evidence, sensory\_context) -> Crest`  
labels: `"alert"`, `"perceptually\_steady"`, `"attentional\_drift"`, `"hostile\_tone"`,  
`"warm\_tone"`, `"sensory\_irregularity"`. Reduces tone/pacing/visual/audio inputs.
2. `memory\_waveform(evidence, contract\_snapshot) -> Crest`  
labels: `"familiar"`, `"unfamiliar"`, `"continuity\_pull"`, `"continuity\_stable"`,  
`"resonant\_recall"`. Reduces working\_memory, conversation\_memory, continuity\_recall.
3. `emotional\_waveform(evidence, contract\_snapshot) -> Crest`  
labels: `"comfort\_bias"`, `"caution"`, `"warmth"`, `"hesitation"`, `"neutral\_affect"`.  
Reduces surface\_reactive\_emotion / deep\_emotional\_state / emotion\_bridge.
4. `prediction\_waveform(prediction\_signal\_obj) -> Crest`  
labels: `"expectation"`, `"surprise"`, `"reframe\_needed"`, `"low\_certainty"`,  
`"steady\_continuation"`. Compresses an existing `PredictionSignal` to a single  
crest — do NOT delete `PredictionSignal`; keep it as the subsurface-only  
detail (see §4) but stop letting it propagate.
5. `symbolic\_waveform(assembly\_result, sensory\_context) -> Crest`  
labels: `"resonance"`, `"contradiction"`, `"novelty"`, `"alignment"`.  
Reduces assembly\_result.law\_bindings / paradoxes / entropy\_state.novelty.
6. `continuity\_waveform(evidence, contract\_snapshot) -> Crest`  
labels: `"thread\_holds"`, `"thread\_slipping"`, `"new\_thread"`, `"context\_drag"`.  
Reduces frame\_continuity, active\_topic, continuity\_bundle.
7. `constraint\_waveform(evidence, projection) -> Crest`  
labels: `"limitation"`, `"strain"`, `"capacity"`, `"steady\_envelope"`.  
Reduces governor\_mode, blocked\_tasks, mem\_available\_mb, runtime\_governor\_axes.  
**\*\*This is the most common leak source — it must compress to a single  
crest label and never propagate raw numbers or governor strings.\*\***

8. ``pressure_waveform(adjusted_axes, pressure_snapshot) -> Crest``  
labels: ``"urgency"`, `"discomfort"`, `"calm"`, `"tension"`.  
Reduces the entire axis pressure map to one crest — DO NOT propagate the pressure_map dict upward anymore.`

Each function must:

- Be pure (no state mutation).
- Take only the slice of evidence relevant to its waveform.
- Return exactly one ``Crest``.
- Have an authorship header.

Also expose a single orchestrator:

```
```python
def emit_subsystem_crests(
    *,
    assembly_result, payload, evidence, contract_snapshot,
    prediction_signal, projection, sensory_context, adjusted_axes,
    pressure_snapshot,
) -> Tuple[Crest, ...]:
    """Run all eight subsystem waveforms and return their crests in canonical
    order. This is the ONLY entry point dce_bridge may use."""
...

---
```

3. NEW MODULE: ``contextual_overlay.py``

****Create:**** ``aurora_core_ai/aurora_internal/dual_strata/contextual_overlay.py``

Implements the Crystal/Overlay distinction. The overlay is a thin transient projection layer that contextualizes activated crystal regions for the current moment WITHOUT mutating the crystal.

```
```python
Authors: Sunni (Sir) Morningstar & Cael Devo
```

```
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any, Dict, List
```

```
@dataclass
class ContextualOverlay:
 """Transient present-state projection onto activated crystals.
```

NEVER permanent. Only fields here that satisfy the  
`embedding\_condition` check (emotional significance, recursive  
reinforcement, continuity importance, symbolic weight, or novelty  
pressure) are eligible for inward crystallization elsewhere.

"""

```
present_anchor: str = "" # the input fragment being held
active_crystals: List[str] = field(default_factory=list) # crystal ids
present_sensory: Dict[str, Any] = field(default_factory=dict) # mic/cam/snap
present_tone: str = "neutral"
interaction_state: str = "present" # present | followup | recall
reinforcement_hint: float = 0.0 # 0..1 — likelihood of inward crystallization
```

```
def to_dict(self) -> Dict[str, Any]: ...
```

```
def embedding_eligible(self) -> bool:
```

```
 """True iff this overlay should be considered for inward crystallization.
```

```
 Inward crystallization itself is performed elsewhere (sedimemory /
 sensory_crystal); this method only signals eligibility upward."""
```

```
 return self.reinforcement_hint >= 0.55
```

```
...
```

The overlay is what consciousness ACTUALLY navigates. The stable crystal stays  
in sedimemory / quasiarch crystal\_engine — it is NOT re-fetched into the  
conscious frame.

---

## 4. REWRITE: `subsurface\_state.py`

Replace `SubsurfaceState` with a crest-bound version. Keep the field name  
`SubsurfaceState` for backward compatibility, but its contents must become:

```
```python
```

```
@dataclass
```

```
class SubsurfaceState:
```

```
    """Recursively generalized subsurface meaning-state. Holds ONLY crests  
    and the converged subsurface crest — no raw subsystem mechanism."""
```

```
    subsurface_crest: Crest          # the converged crest
```

```
    sub_crests: Tuple[Crest, ...]    # the eight micro-crests (frozen)
```

```
    dominant_axis: str               # routing only
```

```
    frame_request: str               # carry through
```

```
    overlay: ContextualOverlay       # present-moment overlay
```

```
    # The fields below are KEPT but marked as `_subsurface_detail` and MUST
```

```
    # NOT be read by surface consumers. They exist only so downward traversal
```

```

# can reach them when consciousness explicitly probes deeper.
_subsurface_detail: Dict[str, Any] = field(default_factory=dict)
...

```

The old raw fields (`pressure\_map`, `salience\_weights`, `recalled\_fragments`, `candidate\_interpretations`, `instability\_markers`, `action\_bias\_candidates`, `contract\_signals`, `prediction`) move INSIDE `\_subsurface\_detail` and become private. `to\_dict()` must NOT serialize `\_subsurface\_detail` into the public upward stream — write it to a separate persisted file (`aurora\_state/subsurface\_detail.json`) so downward traversal can still read it when needed, but the surface daemon never sees it inline.

5. REWRITE: `conscious\_frame.py`

Replace `ConsciousFrame` with:

```

```python
@dataclass
class ConsciousFrame:
 """The conscious crest plus the overlay it navigates. Nothing else."""
 conscious_crest: Crest # the converged conscious crest = response orientation
 subsurface_crest: Crest # carry-up from below for stance/action selection
 overlay: ContextualOverlay # present-moment contextualization
 stance: str # derived from conscious_crest.label
 selected_action: str # derived from conscious_crest.label
 should_speak: bool # derived from conscious_crest.intensity + overlay
 readiness: float # surface-visible readiness (clip01)
 coherence: float # surface-visible coherence (clip01)
 dominant_axis: str # routing
 processing_mode: str # "deliberative" | "reactive" | "blended" | "holding"
 # NO salient_hypotheses, NO explicit_notes (mechanism strings), NO prediction
 # dict, NO contract_signals dict, NO law_bindings, NO comparison_channels,
 # NO origin_systems. Those are subsurface detail.
...

```

Anything previously stuffed into `root\_thought` that was raw mechanism (`law\_bindings`, `comparison\_channels`, `origin\_systems`, `primary\_tension` when it was a raw constraint string) MUST move to `\_subsurface\_detail`.

A new helper `conscious\_frame.summary\_for\_language()` -> str returns ONE human-natural sentence summarizing the crest+overlay — and that is what the language generator consumes. The language generator (`aurora\_articulation.py`

/ `aurora\_language\_faculty.py` / `aurora\_grammar\_engine.py`) must stop reading `salient\_hypotheses`, `explicit\_notes`, `contract\_signals`, `prediction`, or `root\_thought` mechanism fields. If they need detail, they trigger downward traversal (see §7) — they do NOT inline-read the dict.

---

## 6. REWRITE: `dce\_bridge.py`

`DualStrataBridge.build\_snapshot` becomes the **crest convergence orchestrator**, not a packet aggregator.

New flow (replace `\_build\_subsurface\_state` and `\_build\_conscious\_frame`):

...

```
build_snapshot(assembly, payload, payload_type, evidence,
 contract_snapshot, requested_frame, thought_intent):
```

```
1. Build the present overlay (transient, may crystallize inward later)
overlay = build_contextual_overlay(payload, evidence, contract_snapshot,
 sensory_context=...)
```

```
2. Build the prediction detail object (kept SUBSURFACE-ONLY)
prediction_signal = build_prediction_signal(...)
```

```
3. Emit subsystem crests — every one of the eight waveforms
sub_crests = emit_subsystem_crests(
 assembly_result=assembly, payload=payload, evidence=evidence,
 contract_snapshot=contract_snapshot,
 prediction_signal=prediction_signal,
 projection=evidence.get("subsurface_projection") or {},
 sensory_context=getattr(assembly, "sensory_context", {}) or {},
 adjusted_axes=getattr(assembly, "adjusted_axes", {}) or {},
 pressure_snapshot=evidence.get("pressure_snapshot") or {},
)
```

```
4. Subsurface convergence — converge the 8 sub-crests into ONE
subsurface_crest = converge_crests(sub_crests, mode="subsurface")
```

```
5. Build SubsurfaceState (crest-only public, detail private)
subsurface = SubsurfaceState(
 subsurface_crest=subsurface_crest,
 sub_crests=sub_crests,
 dominant_axis=subsurface_crest.axis,
```

```

frame_request=requested_frame,
overlay=overlay,
_subsurface_detail={
 # everything that used to be public goes here, kept for
 # downward traversal, written to subsurface_detail.json
 "prediction_signal": prediction_signal.to_dict(),
 "pressure_map": ...,
 "salience_weights": ...,
 "candidate_interpretations": ...,
 "instability_markers": ...,
 "action_bias_candidates": ...,
 "contract_signals": ...,
 "recalled_fragments": ...,
 "law_bindings": getattr(assembly, "law_bindings", []),
 "comparison_channels": ...,
 "origin_systems": ...,
},
)

6. Surface convergence — consume only subsurface_crest + overlay +
live sensory state, produce the conscious_crest.
conscious_crest = converge_for_surface(
 subsurface_crest=subsurface_crest,
 overlay=overlay,
 coherence=getattr(assembly, "coherence", 0.45),
 user_turn_present=bool(...),
)

7. Derive stance/action/should_speak FROM the conscious_crest label —
not from a list of hypotheses.
stance, action, should_speak, processing_mode = derive_surface_behavior(
 conscious_crest, subsurface_crest, overlay, coherence=...
)

conscious = ConsciousFrame(
 conscious_crest=conscious_crest,
 subsurface_crest=subsurface_crest,
 overlay=overlay,
 stance=stance,
 selected_action=action,
 should_speak=should_speak,
 readiness=..., # derived from conscious_crest.intensity + overlay
 coherence=clip01(getattr(assembly, "coherence", 0.45)),
 dominant_axis=conscious_crest.axis,

```

```

 processing_mode=processing_mode,
)

 snapshot = DualStrataSnapshot(...)
 self.persist(snapshot)
 self.persist_subsurface_detail(subsurface._subsurface_detail) # new
 return snapshot
...

```

Add helpers:

- ``converge_crests(sub_crests, mode)`` -> `Crest`` — picks the dominant or builds a higher-order generalized label (e.g. multiple high-intensity caution-shaped sub-crests → ``"caution"``; mixed familiarity + uncertainty → ``"intuitive_caution"``).
- ``converge_for_surface(subsurface_crest, overlay, coherence, user_turn_present)`` -> `Crest`` — applies the surface compression pass that turns subconscious meaning + live context into a single conscious crest like ``"explain"`, `"clarify"`, `"comfort"`, `"hold"`, `"contextualize"`, `"reframe"`.`
- ``derive_surface_behavior(...)`` — pure label → stance/action/mode lookup, no mechanism inspection. Replace the cascade of ``if any(item.get("label") == ...)`` blocks currently in ``_build_conscious_frame``.

The ``persist`` method must keep writing ``dual_strata_snapshot.json`` so the surface daemon's existing file watcher keeps working — but the `conscious_frame` section is now lean. Mechanism goes to the new sibling file ``subsurface_detail.json`` only.

---

## ## 7. DOWNWARD TRAVERSAL

Add a small module so the conscious layer can deliberately reach down when the user asks for elaboration:

**\*\*Create:\*\*** ``aurora_core_ai/aurora_internal/dual_strata/downward_traversal.py``

```

```python
# Authors: Sunni (Sir) Morningstar & Cael Devo

```

```

from pathlib import Path
import json

```

```

def expand_crest(state_dir: Path, crest_label: str, depth: int = 1) -> dict:
    """When consciousness needs to elaborate, read subsurface_detail.json and
    return only the slice relevant to the named crest at the requested depth.

```

depth=1: the sub-crests that produced this crest
depth=2: the raw subsystem inputs behind those sub-crests
depth=3: the underlying numeric mechanism (rarely needed for language)
""

...

This is the ONLY supported path from surface into subsurface mechanism. No direct dict-walking of `\_subsurface\_detail` from language modules.

8. ADAPT CONSUMERS

These files currently read fields that no longer exist on the `conscious_frame`. Update each to read either the new crest fields or — when they truly need detail — call ``downward_traversal.expand_crest``:

- ``aurora_core_ai/aurora.py`` lines ~17428–17496: ``_read_live_dual_strata_runtime`` and ``_refresh_live_dual_strata_runtime``. Replace ``root_thought`` reads with ``conscious_crest`` / ``overlay`` reads. ``processing_mode`` and ``reactive_signal`` derivations now come from the crest label + intensity.
- ``aurora_core_ai/aurora_surface_daemon.py`` line 563:
``payload.get("conscious_frame", {}).get("salient_hypotheses")`` → replace with ``[conscious_crest.label] + [c.label for c in subsurface_crest_subs]`` resolved via ``expand_crest`` when concepts are needed for the continuity packet.
- ``aurora_core_ai/aurora_consciousness_engine.py`` line 1173: no signature change to ``build_snapshot``; just update any post-snapshot reads that grab raw fields.
- Mirror all of the above into the root-level duplicates: ``aurora.py``, ``aurora_consciousness_engine.py``, ``aurora_surface_daemon.py``, ``aurora_internal/dual_strata/*`` — the project has two parallel trees and both must stay in sync.

9. PROJECTION ALIGNMENT

``subsurface_projection.py::build_subsurface_projection`` currently uses heuristic axis/keyword gating to soften signals before the surface sees them. Re-anchor it so it consumes the new ``SubsurfaceState.subsurface_crest`` directly:

- ``intuition_signals[*].label`` should now equal sub-crest labels, not be re-derived from ``repair_issue`` keyword matching.

- `active\_effects` becomes the crest-natural-language fragments only.
- `surface\_guidance` is derived from the converged subsurface_crest label, not from a chain of axis/repair_phase if-branches.

Keep the file path `aurora\_state/subsurface\_projection.json` stable — the surface daemon (`aurora\_surface\_daemon.py` line 29) reads it.

10. HARD CONSTRAINTS — NON-NEGOTIABLE

When implementing the above, every changed or created Aurora module MUST:

1. Begin with `# Authors: Sunni (Sir) Morningstar & Cael Devo`
2. Keep `aurora\_` naming for any new modules
3. Use canonical axis symbols `X`, `T`, `N`, `B`, `A` only — never alternate names
4. Honor the leverage scalar boundary: do not import or read internal scalar state inside any of the new waveforms; only `band\_position` and `PhaseNudge` deltas may cross (already enforced in `aurora\_leverage\_scalar.py`, do not loosen)
5. Maintain the 25-NonComp / 625-slot atomic basis where pressure references are used inside the new pressure_waveform
6. Use canonical I-State polarities `IS/ISNT`, `CAN/CANT`, `DO/DONT`, `SAW/SAUNT`, `DID/DIDNT` if any crest label needs to reference them
7. Honor the re-entry loop sequence
`STATE → EXPRESSION → RE-ENTRY → RECONCILIATION → UNDERSTANDING`
anywhere the reflexive interpreter touches the new crest pipeline

11. ACCEPTANCE TEST

After implementation, the following must all be true:

- [] `ConsciousFrame.to\_dict()` contains no field whose value is a list of `{label, rationale, source, tags, confidence}` shaped dicts
- [] `ConsciousFrame.to\_dict()` contains no `pressure\_map`, `salience\_weights`, `instability\_markers`, `candidate\_interpretations`, `action\_bias\_candidates`, `contract\_signals`, or `prediction` dict — only crests + overlay + derived simple fields
- [] `SubsurfaceState.to\_dict()` contains only `subsurface\_crest`, `sub\_crests`, `dominant\_axis`, `frame\_request`, and `overlay` — no `\_subsurface\_detail`
- [] `aurora\_state/subsurface\_detail.json` exists after a turn and contains all the formerly-public mechanism fields

- [] Language modules read ``summary_for_language()`` or call ``downward_traversal.expand_crest``; no language module dict-walks raw mechanism fields
- [] ``aurora_surface_daemon.py`` continues to write ``continuity_packet`` successfully without referencing ``salient_hypotheses`` as a structured list
- [] Run ``test_full_reasoning_pipeline.py`` and confirm a full turn produces a ``ConsciousFrame`` whose ``conscious_crest.label`` is one of the documented crest labels and whose response no longer contains internal mechanism strings (no ``intent=`, `topic=`, `axis=`, `affect=`, `certainty=`, or `Predicted continuation:`` prefixes leaking through)

12. WHAT THIS FIXES

When this is done:

- Constraint mechanics will no longer surface as response content
- Sensory scaffolding will no longer leak as token-style descriptors
- Predictive machinery will no longer narrate itself
- Aurora will speak from the converged crest only, with downward traversal available on demand for genuine elaboration
- The crystal/overlay distinction becomes operational: stable concepts persist, transient observation modulates, and only reinforced overlays crystallize inward (handled in a subsequent sedimemory wiring pass — out of scope for this directive)

...